

Programming

Lecture 04 — Iterations

While, for, and the patterns that live inside every loop



Dr. Ramon Winterhalder

Human-Centered Artificial Intelligence · A.Y. 2026/27



UNIVERSITÀ
DEGLI STUDI
DI MILANO



Where we left off

Lecture 03, in four lines

input() always returns text

Wrap it in `int()` the moment you need a number.

f-strings

`f"{value:>8.2f}"` — line the columns up, two decimals.

if / elif / else

Exactly one branch runs, and they are checked in order.

Most specific first

A branch that can never be reached is a silent bug.

Your programs can now ask and choose. They still do each thing exactly once. Today they learn to repeat — and that is where the work starts paying.

Sheet 2 ended with "add a second product and see how much you had to touch". Sheet 3 ended with branches doubling. Both answers start here.

By the end of today you can...

- write a while loop, and say what makes it stop;
- recognize the accumulator pattern, and write it without thinking;
- write a for loop over `range()`, and get the endpoints right;
- choose between while and for, and justify the choice;
- trace a nested loop by hand and predict its output.

Tracing is the skill that pays. A loop you can trace is a loop you can debug — and in the exam there is no Python to run it for you.



while

Conditions · accumulators · break and continue

The problem

You already have it

Lecture 02, sheet 2

```
notebooks = 3 * 2.40
pens = 2 * 1.15

net = notebooks + pens
```

Two products, four lines. Ten products, twenty lines.
A hundred?

And you cannot write it at all...

- ...when you do not know how many there are until the program runs;
- ...when the user decides when to stop;
- ...when you are waiting for a condition to become true.

A loop says: do this again, and here is what makes you stop.

Everything today is one of two shapes. while: repeat until something changes. for: repeat a known number of times.

while

Repeat while something is true

```
i = 1                # 1. set up
while i <= 5:        # 2. the condition
    print(i)         # 3. the body
    i = i + 1        # 4. the change
```

1 set up

give the variable its starting value, before the loop

2 condition

checked before every pass. True: go on. False: stop

3 body

what happens each time. Indented, like an if

4 change

something must move towards making the condition False

Miss step 4 and the loop never ends. Miss step 1 and it never starts.

Output: 1 2 3 4 5 – five passes, then i is 6 and the condition fails

Trace it by hand

Your algorithm from week 1, now in Python

```
n = 4
total = 0
i = 1

while i <= n:
    total = total + i
    i = i + 1

print(total)
```

pass	i	i <= 4	total	i after
before	1	—	0	—
1	1	True	1	2
2	2	True	3	3
3	3	True	6	4
4	4	True	10	5
5	5	False	10	—

This is exactly sheet 1, A3 — the pseudocode you traced on paper in week one. Same table, same answer, now with a program that runs it.

The last row matters: the condition is checked a sixth time, fails, and the body never runs. A loop always tests once more than it repeats.

The accumulator pattern

Name it now — you will use it every week

```
total = 0                # start empty

while ...:
    total = total + x    # add this one in

print(total)            # read it off
```

- A variable that lives OUTSIDE the loop and grows INSIDE it.
- Starts at the neutral value: 0 for a sum, 1 for a product.
- Counting is the same pattern: `count = count + 1`.
- `total += x` is the shorthand from lecture 2 — same thing.

Declared outside, updated inside, read after. Get that shape wrong and the loop resets its own answer every pass.

```
total = 0                total = 0
while ...:                while ...:
    total = total + x      total = 0 + x    # the classic bug
```

Three ways a loop goes wrong

1

It never starts

The condition is False at the first check.

```
i = 10  
while i < 5:
```

2

It never stops

Nothing in the body moves towards False.

```
i = 1  
while i < 5:  
    print(i)
```

3

It runs the wrong number of times

Off by one, almost always at the end.

```
while i < n:   vs   while i <= n:
```

Number 2 is the only one that announces itself — by never finishing. The other two just give you a wrong answer.

When it never stops

It will happen today

```
x = 10
while x > 0:
    print(x)
    x = x + 1      # should be x - 1
```

Sheet 1, A5 — the one you fixed on paper. Here it is for real.

How to stop it

Notebook

the square Stop button, or Kernel → Interrupt

Terminal

Ctrl + C

Either way

the program stops where it was.
Nothing is broken.

A notebook cell showing [*] forever is not frozen — it is doing exactly what you told it to, very fast, for ever. Interrupt it and read your condition.

Print inside the loop while you are debugging. A loop that prints nothing tells you nothing.

Worked example

Numbers until someone says stop

The plan

```
# a running total, and a count
# ask for a number
# while it is not 0:
#     add it in, count it
#     ask again
# print the total and the average
```

```
total = 0
count = 0

value = int(input("Number (0 to stop): "))

while value != 0:
    total += value
    count += 1
    value = int(input("Number (0 to stop): "))

print(f"Total: {total}")
print(f"Average: {total / count:.2f}")
```

Note the question is asked twice — once before the loop, once at the end of the body. That is the shape of every loop that reads until a stopping value.

break and continue

Two escape hatches

break — leave the loop now

```
while True:
    value = int(input("n: "))
    if value == 0:
        break
    total += value
```

No duplicated question. The exit is in the middle, which is exactly where it belongs.

continue — skip to the next pass

```
while i <= 20:
    i += 1
    if i % 2 != 0:
        continue
    print(i)                # even only
```

Careful: put the change BEFORE the continue, or you skip it and the loop hangs.

while True: with a break is a normal, readable loop — not a trick.

Use continue rarely. An if around the body usually reads better.

Exercises — Block A

Sheet 04, part A · 40 minutes · trace before you run

- Trace four while loops and give the final values.
- Say which of six loops never start, never stop, or are off by one.
- Write the accumulator that sums the numbers from 1 to n.
- Write a loop that keeps asking until the answer is valid.
- Find the bug in a loop that resets its own total.

Draw the trace table. Four columns and six rows finds a bug faster than twenty minutes of staring, and it is what the exam will ask you to do.



for and range

Counted repetition · nesting · patterns

for

When you know how many times

```
for i in range(5):  
    print(i)
```

```
0  
1  
2  
3  
4
```

- The counter is created for you — no setting up, no incrementing.
- It cannot run away: range decides how many passes there are.
- Steps 1 and 4 of the while loop are done for you. Only the body is yours.
- It starts at 0 and stops BEFORE 5. Five numbers, none of them 5.

Same five passes as the while loop, with two fewer things to get wrong.

range()

Three forms, one rule

written	gives	
range(5)	0 1 2 3 4	start at 0, stop before 5
range(1, 6)	1 2 3 4 5	start at 1, stop before 6
range(1, 10, 2)	1 3 5 7 9	...in steps of 2
range(10, 0, -1)	10 9 8 ... 1	a negative step counts down
range(0)	(nothing)	a loop that runs zero times is legal

The stop value is never included. range(1, 6) stops before 6, so it ends at 5.

```
for i in range(1, n + 1):    # the numbers 1 to n, inclusive – the +1 is the point
```

while or for?

Ask one question

Do I know, before the loop starts, how many times it will\should run?

Yes → for

- five times, ten times, n times;
- once for every number in a range;
- once for every item in something (lecture 6).

No → while

- until the user types something;
- until the answer is good enough;
- until a condition changes.

Anything a for loop can do, a while loop can do — with more code and more ways to get it wrong. Prefer for when the question above says yes.

Worked example

A times table — a loop inside a loop

The plan

```
# for each row 1..5
#     for each column 1..5
#         print the product, padded
#     end the line
```

```
for row in range(1, 6):
    for col in range(1, 6):
        print(f"{row * col:>4}", end="")
    print()
```

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25

- The inner loop runs completely for each single pass of the outer one.
- `end=""` keeps the row on one line — lecture 3 earning its keep.
- The bare `print()` at the outer level ends the row.

Nested loops

How to think about them

```
for i in range(3):  
    for j in range(4):  
        print(i, j)
```

- The outer loop runs 3 times.
- For each of those, the inner runs 4 times.
- So the body runs $3 \times 4 = 12$ times.
- The inner counter goes all the way round before the outer one moves.

i=0: j=0 1 2 3

i=1: j=0 1 2 3

i=2: j=0 1 2 3

Use different names for the two counters. i and j is the convention; i and i is a bug you will spend an hour on.

Cost adds up fast: two nested loops over n items do n^2 passes. For $n = 1000$ that is a million. Worth knowing now, properly true in lecture 10.

Worked example

Is it a prime?

```
n = int(input("A number: "))

is_prime = True

for d in range(2, n):
    if n % d == 0:
        is_prime = False
        break

if is_prime and n > 1:
    print(f"{n} is prime")
else:
    print(f"{n} is not prime")
```

- A flag: one bool that remembers what the loop found.
- Set it to True, and let the loop disprove it.
- break the moment you know — there is nothing left to learn.
- $n \% d == 0$ is the divisibility test from lecture 2.
- The $n > 1$ guard catches 1 and 0, which divide by nothing and are not prime.

The flag-plus-break shape answers every "is there any...?" question you will ever write.

Exercises — Block B

Sheet 04, part B · 40 minutes · plan in comments first

Write

- A countdown, and a times table for one number.
- A factorial, with the right starting value.
- A triangle of stars, then a pyramid.
- The largest of n numbers typed in one by one.

Read and predict

- Six range() calls: what do they produce?
- Four nested loops: how many lines, and what?
- Three loops that are off by one — say by how much.

The largest-of-n question is the accumulator again, with "bigger than" instead of "plus". You designed it on paper in week one — sheet 1, worked example 2.

Today in one slide

while repeats until something changes

Set up, condition, body, change. Miss the change and it never stops.

for repeats a known number of times

The counter is made for you and cannot run away.

range stops BEFORE its end value

`range(1, n+1)` is how you get 1 to n inclusive.

The accumulator

Declared outside, updated inside, read after.

A flag remembers what a loop found

Set it True, let the loop disprove it, break when you know.

Trace, do not stare

Four columns and six rows beats twenty minutes of reading.

Before next time

- Finish sheet 04. Solutions go up after we discuss them.
- Deliberately write one infinite loop and interrupt it. Get it over with in private.
- Read chapter 1.3 of Sedgewick, Wayne & Dondero.
- Bring the laptop.

Next: Lecture 05 — strings. You have been printing text all along; next week you take it apart.